

```

1 I!:{ooooooooo-----
2 ___oo_____ooooo___ooooo___
3 ___oo_____oo_____o_oo___oo_
4 ___oo_____ooooooooo_oo___oo_
5 ___oo_____oo_____oo___oo_
6 ___oo_____ooooo___ooooo_o_
7 -----}
8 x!:{
9 TEA OS v1.1.0 is Ready...}
10 v:vBanner
11 L:lPREAMBLE_START
12 Y:vBanner
13 V@:dvLOG_start
14 V@:dvABOUT_TOS:{
15 #---[ABOUT TEA OS]
16
17 NAME: TEA Operating System
18 (also "TEA OS" or just "TOS")
19
20 ARCHITECT: J. Willrich Lutalo <jwl@nuchwezi.com>
21
22 SINCE: 9th APRIL, 2026
23
24 INTENT: To manifest a true general purpose, lightweight, minimal but powerful operating
        system that can essentially just fit inside a single string; to see what the limits
        of the mathematics of TRANSFORMATICS might be, and what we surely could accomplish
        with its proper use via modern portable computers.
25
26 ABOUT: TEA OS is implemented using the Transforming Executable Alphabet (TEA)
        programming language. It is based on a specification first laid out in the TEA TAZ.
        It is a project originating from UGANDA.
27
28 PROJECT HOME: Nuchwezi Research | https://tea.nuchwezi.com}
29 V@:dvOSName:{TEA OS}
30 V@:dvOSName_short:{TOS}
31 V@:dvOSVersion:{1.1.0}
32 V:vSLOGAN:{TEA OS! T can fit in a string!}
33 I!:{-----}
34 V:vDELIM
35 V@:dvDELIM
36 I!:{
37 =o==o==o==o==o==o==o==
38 }
39 V:vDELIM2
40 V@:dvDELIM2
41 V:vUSERNAME:{user}
42 V@:dvLastPS:{}
43 V@:dvLastCommand:{}
44 V@:dvLastOutput:{}
45 Y@:dvFileList
46 I:{.}
47 V@:dvFileList
48 Y@:dvProgramsList
49 I:{run}
50 V@:dvProgramsList
51 Y@:dvUIMODE_MINI
52 I:{1}
53 V@:dvUIMODE_MINI
54 V:fPROMPT_DEFAULT:"
55 # FIRST: IMPORTANTLY...
56 v:vMSG #stores AI at invocation
57 y@:dvDELIM|v:vD2
58 y:vMSG|x!:{ }|v:vMSG #affix space at prompt end
59 i!:{ } | g*:{
60 }:vD2:vMSG

```

```

61 v:vMSG
62 Y@:dfLogAI|v:f # a stealth import :)
63 y:vMSG|e*:f
64 #then show log+latest prompt
65 Y@:dvLOG
66 i: #returns input..
67 "
68 Y:fPROMPT_DEFAULT
69 V@:dfPROMPT_DEFAULT
70 V:fPROMPT:"
71 # FIRST: IMPORTANTLY...
72 v:vMSG #stores AI at invocation
73
74 #check if we should instead be using default ui
75 y@:dvUIMODE_MINI | f!:^0$:lN_UIMODE_MINI
76 y@:dfPROMPT_DEFAULT|v:fPROMPT_DEFAULT
77 y:vMSG | E*:fPROMPT_DEFAULT
78 q!: # no need to proceed..
79 l:lN_UIMODE_MINI
80
81 #THEN imports...
82 y@:dfLastIO|v:fLastIO
83 y@:dfMakePS|v:fMakePS
84
85 y@:dvDELIM|v:vD2
86 y:vMSG|x!:{ }|v:vMSG #affix space at prompt end
87 i!:{ } | g*:{
88 } :vD2:vMSG
89 v:vMSG
90 Y@:dfLogAI|v:f # a stealth import :)
91 y:vMSG|e*:f
92 #then show log+latest prompt
93 #Y@:dvLOG
94 E*:fMakePS|v:vNEXTPS
95 E*:fLastIO #fetch last I/O
96 x!:{
97 > }|#x*!:vNEXTPS
98 i: #returns input..
99 "
100 Y:fPROMPT
101 V@:dfPROMPT
102 V:fLogin:"
103 #import some functions from database...
104 y@:dfPROMPT_DEFAULT|v:fPROMPT
105 y@:dfLogAI|v:fLogAI
106 y@:dfLogO|v:fLogO
107 y@:dfGreetUser|v:fGreetUser
108
109 # show OS banner
110 y@:dvLOG_start | v:vSCREEN
111 e*:fLogAI
112
113 #greet user via stored name..
114 y!@:dvUSERNAME | f!:^0$:lGREET_USER
115 #Username not yet set, prompt and store
116 l:lSET_USERNAME
117 I!:{
118 Please set a Username:}|v:vPromptMSG
119 e*:fLogAI
120 y:vSCREEN | x*!:vPromptMSG | i:
121 t!.:|v:|v!|f!:^0$:lSET_USERNAME
122 y:|V@:dvUSERNAME # store in database
123 e*:fLogAI
124
125 l:lGREET_USER
126 e*:fGreetUser|v:vGreetings

```

```

127 e*:fLog0 | e*:fPROMPT
128 "
129 V:fGreetUser:"
130 y@:dvUSERNAME
131 x:{
132 Hello, }|x!:{. Welcome to TOS!}
133 "
134 Y:fGreetUser
135 V@:dfGreetUser
136 V:fOSHHandle:"
137 y@:dvOSName | v:vN
138 y@:dvOSVersion | v:vV
139 g*:{ v}:vN:vV
140 "
141 Y:fOSHHandle
142 V@:dfOSHHandle
143 V:fMakePS:"
144 z.:TIME |v:vT
145 y@:dvOSName_short | v:vPWD
146 v:vD:{>}
147 g*:{/}:vPWD:vT:vD
148 "
149 Y:fMakePS
150 V@:dfMakePS
151 V:fLogIO:"
152 #load+execute function from database!
153 y@:dfLastIO|e:
154 v:vLastIO
155 y@:dvLOG | v:vLOG
156 g*:{
157 }:vLOG:vLastIO | v@:dvLOG
158 "
159 V:fLogAI:"
160 v:vAI
161 y@:dvLOG | v:vLOG
162 g*:{
163 }:vLOG:vAI | v@:dvLOG
164 #return AI
165 y:vAI
166 "
167 Y:fLogAI
168 v@:dfLogAI
169 V:fLog0:"
170 v@:dvLastOutput
171 v@:dvDATABUFFER #also stash into special data buffer
172 Y@:dvLastOutput
173 "
174 V:fLastIO:"
175 y@:dvLastPS | v:vPS
176 y@:dvLastCommand | v:vC
177 y@:dvLastOutput | v:vO
178 g*:{}:vPS:vC|v:vCC
179 g*:{
180 }:vCC:vO
181 "
182 y:fLastIO
183 v@:dfLastIO
184 L:1PREAMBLE_END
185 L:1SYS_UTILITY_START
186 V:fFixURL:"
187 v:vURL|z:
188 f:{^https?://}:1FINE
189 y:vURL|x:{https://}
190 v:vURL
191 l:1FINE
192 y:vURL

```

```

193 "
194 V:fResetLog:"
195 y@:dvLastOutput | v@:dvLastOutput_backup
196 c!@:dvLOG
197 c*:vLOG
198 C!@:dvLastOutput
199 y@:dfGreetUser|E:|v:vGreetings
200 y@:dvLOG_start
201 x*!:vGreetings
202 "
203 Y:fResetLog
204 V@:dfResetLog
205 V:fGetLatestOutputBuffer:"
206 y@:dvLastOutput_backup
207 f!:~$:1N_USED_BACKUP
208 y@:dvLastOutput
209 l:1N_USED_BACKUP
210 c@:dvLastOutput_backup
211 "
212 Y:fGetLatestOutputBuffer
213 V@:dfGetLatestOutputBuffer
214 L:1SYS_UTILITY_END
215 L:1SYS_HELP_START
216 V:fGetUIMenu:"
217 y@:dfOShandle|e:|x!:{ INTERFACE System Commands:
218 uimini
219 uilog}
220 v:vMSG
221 y@:dvDELIM2|v:vD
222 g*:{}:vD:vMSG:vD
223 "
224 V:fGetFileCMDMenu:"
225 y@:dfOShandle|e:|x!:{ FILE System Commands:
226 create FILENAME
227 init FILENAME
228 write FILENAME
229 (rename | rn | mv | move) FILENAME NEWNAME
230 (copy|clone) FILE1 FILE2
231 exists FILENAME
232 find PATTERN
233 (show | cat | read) FILENAME
234 export FILENAME
235 (del | delete | rm) FILENAME
236 (ls | list | dir | files) PATTERN}
237 v:vMSG
238 y@:dvDELIM2|v:vD
239 g*:{}:vD:vMSG:vD
240 "
241 V:fGetProgrammingMenu:"
242 y@:dfOShandle|e:|x!:{ PROGRAMMING System Commands:
243 (runnable | register | add-program) FILENAME
244 (isrunnable | canrun) FILENAME
245 (run | exec | execute) FILENAME
246 (del-prog | delete-program | unregister) FILENAME
247 (update | update-program) FILENAME
248 PROGRAM
249 (help | man | whatis | info | about) PROGRAM}
250 v:vMSG
251 y@:dvDELIM2|v:vD
252 g*:{}:vD:vMSG:vD
253 "
254 V:fGetUtilityMenu:"
255 y@:dfOShandle|e:|x!:{ UTILITY System Commands:
256 clear | reset
257 date
258 echo MESSAGE

```

```

259 exit
260 export
261 (fetch | web) [URL] [FILENAME]
262 logout
263 math EXPRESSION
264 print
265 tea CODE
266 time
267 timestamp
268 username NAME
269 whoami | name}
270 v:vMSG
271 y@:dvDELIM2|v:vD
272 g*:{}:vD:vMSG:vD
273 "
274 V:fGetHelpMenu:"
275 y@:df0SHandle|e:|x!:{ HELP System Commands:
276 about
277 (help | man | whatis | info | about) [NAME]
278 help-file
279 help-help
280 help-prog
281 help-ui
282 help-util}
283 v:vMSG
284 y@:dvDELIM2|v:vD
285 g*:{}:vD:vMSG:vD
286 "
287 V:fGetAboutTOS:"
288 y@:df0SHandle|e:|x:{ABOUT: }
289 v:vMSG
290 y@:dvABOUT_TOS|x*:vMSG|v:vMSG
291 y@:dvDELIM2|v:vD
292 g*:{}:vD:vMSG:vD
293 "
294 V:fGetHelpOn:"
295 #asumes topic is passed via AI
296 v:vHelpTOPIC
297
298 #some imports...
299 y@:dfGetFilenameProtocolMSG|v:fGetFilenameProtocolMSG
300 E*:fGetFilenameProtocolMSG | v:vFILENAME_PROTOCOL
301
302 #set default..
303 r@:{That help topic [#vHelpTOPIC#] is not understood or supported. Try the 'help'
      command to see what help commands there are. For example 'help whatis'}|v:vMSG
304
305 y:vHelpTOPIC
306
307 #> help about
308 f!:^about$:lN_h0
309 v:vMSG:{about <~ Displays basic info about the currently running version of the TEA
      Operating System.}
310 j:lh1_present
311 l:lN_h0
312
313 #> help whatis
314 f!:^whatis$:lN_h1
315 v:vMSG:{whatis NAME <~ Displays help info about a given system command or user program
      .}
316 j:lh1_present
317 l:lN_h1
318
319 #> help man
320 f!:^man$:lN_h1b
321 v:vMSG:{man NAME <~ Displays help info about a given system command or user program. It

```

```

        is just another alias for 'whatis' or 'help'}
322 j:lh1_present
323 l:1N_h1b
324
325 #> help help
326 f!:^help$:1N_h2
327 v:vMSG:{Present menu for user to access help about all available [essential] commands
        based on category.}
328 j:lh1_present
329 l:1N_h2
330
331 f!:^help-file$:1N_h3
332 v:vMSG:{Display all available commands in the FILE I/O category or sub-system.}
333 j:lh1_present
334 l:1N_h3
335
336 f!:^help-prog$:1N_h4
337 v:vMSG:{Display all available commands in the PROGRAMMING category or sub-system.}
338 j:lh1_present
339 l:1N_h4
340
341 f!:^help-util$:1N_h5
342 v:vMSG:{Display all available commands in the UTILITIES category or sub-system.}
343 j:lh1_present
344 l:1N_h5
345
346 f!:^help-help$:1N_h6
347 v:vMSG:{Display help on how to use the HELP facility or sub-system.}
348 j:lh1_present
349 l:1N_h6
350
351 f!:^{(clear|reset)}$:1N_hu0
352 v:vCMD
353 r@:{#vCMD# <~ Resets and clears the current TEA OS session log since login.} | v:vMSG
354 j:lh1_present
355 l:1N_hu0
356
357 f!:^{(fetch|web)}$:1N_hu1
358 v:vCMD
359 r@:{#vCMD# URL <~ fetch and output contents of the specified [online/network] resource.
360
361 #vCMD# URL FILENAME <~ fetch and save contents at URL into file record named FILENAME.
        #vFILENAME_PROTOCOL#} | v:vMSG
362 j:lh1_present
363 l:1N_hu1
364
365 f!:^tea$:1N_hu2
366 v:vMSG:{tea CODE.. <~ take everything following the keyword 'tea' as a TEA program, run
        and output the result: It's a simple way to write and run basic TEA programs (TEA-
        one-liners) in TOS.}
367 j:lh1_present
368 l:1N_hu2
369
370 f!:^math$:1N_hu3
371 v:vMSG:{math EXPRESSION <~ execute basic mathematics on the command-line as specified
        in EXPRESSION.}
372 j:lh1_present
373 l:1N_hu3
374
375 f!:^date$:1N_hu4
376 v:vMSG:{date <~ Show the current date.}
377 j:lh1_present
378 l:1N_hu4
379
380 f!:^time$:1N_hu5
381 v:vMSG:{time <~ Show the current time.}

```

```

382 j:lh1_present
383 l:lN_hu5
384
385 f!:^timestamp$:lN_hu6
386 v:vMSG:{timestamp <~ Show the current timestamp.}
387 j:lh1_present
388 l:lN_hu6
389
390 f!:^print$:lN_hu7
391 v:vMSG:{print <~ Show or preview current output buffer contents (usually, output of
      last command). After clearing log, but also after exit/system shutdown/login also
      shows final output before the log was reset.}
392 j:lh1_present
393 l:lN_hu7
394
395 f!:^export$:lN_hu8
396 r@:{export <~ Take the output of the last command or rather, current contents of the
      output buffer and export that outside the operating system (also terminates the
      operating system session).
397
398 export FILENAME <~ Exit the system, outputting the contents of given file. #
      vFILENAME_PROTOCOL#} | v:vMSG
399 j:lh1_present
400 l:lN_hu8
401
402 f!:^exit$:lN_hu9
403 v:vMSG:{exit <~ Ends the TOS session returning nothing.}
404 j:lh1_present
405 l:lN_hu9
406
407 f!:^logout$:lN_hu10
408 v:vMSG:{logout <~ Resets username and system logs and ends the TOS session returning
      nothing.}
409 j:lh1_present
410 l:lN_hu10
411
412 f!:^username$:lN_hu11
413 v:vMSG:{username NAME <~ Overrides current username with specified value.}
414 j:lh1_present
415 l:lN_hu11
416
417 f!:^help-ui$:lN_h12
418 v:vMSG:{Display all available commands in the USER INTERFACE category or sub-system.}
419 j:lh1_present
420 l:lN_h12
421
422 f!:^uimini$:lN_h13
423 v:vMSG:{uimini <~ Configure TEA OS interface to render using mini-terminal mode.
      Setting persists across logins until changed. This mode is suitable for environments
      such as using TOS on some Desktop Computer Systems.}
424 j:lh1_present
425 l:lN_h13
426
427 f!:^uilog$:lN_h14
428 v:vMSG:{uilog <~ Configure TEA OS interface to render using long/log-terminal mode.
      Setting persists across logins until changed. This mode is suitable for environments
      such as using TOS on the TEA MOBILE IDE.}
429 j:lh1_present
430 l:lN_h14
431
432 f!:^whoami$:lN_h15
433 v:vCMD
434 r@:{#vCMD# <~ Display the NAME of currently active USER.} | v:vMSG
435 j:lh1_present
436 l:lN_h15
437

```

```

438 f!:^create$:lN_h16
439 r@:{create FILENAME <~ Creates blank file with name FILENAME. #vFILENAME_PROTOCOL#}|v:
    vMSG
440 j:lh1_present
441 l:lN_h16
442
443 f!:{^(ls|list|dir|files)$}:lN_h17
444 v:vCMD
445 r@:{#vCMD# <~ Display list of ALL available files.
446
447 #vCMD# PATTERN <~ Display list of ONLY files matching regular expression PATTERN.}
448 | v:vMSG
449 j:lh1_present
450 l:lN_h17
451
452 f!:^init$:lN_h18
453 r@:{init FILENAME <~ Create a file using the given name and the currently available
    input/last-output (or the PRINT buffer) or blank if none. #vFILENAME_PROTOCOL#}|v:
    vMSG
454 j:lh1_present
455 l:lN_h18
456
457 f!:^write$:lN_h19
458 r@:{write FILENAME <~ Write the contents of the current available input/last-output (or
    the PRINT buffer) or blank if none, to a file with the specified name. If the file
    already exists, essentially overwrites its contents. #vFILENAME_PROTOCOL#} | v:vMSG
459 j:lh1_present
460 l:lN_h19
461
462 f!:{^(show|cat|read)$}:lN_h20
463 v:vCMD
464 r@:{#vCMD# FILENAME <~ Preview the contents of given file. #vFILENAME_PROTOCOL#} | v:
    vMSG
465 j:lh1_present
466 l:lN_h20
467
468 f!:^echo$:lN_h21
469 v:vMSG:{echo MESSAGE <~ Simply print MESSAGE as is.}
470 j:lh1_present
471 l:lN_h21
472
473 f!:{^(find)$}:lN_h22
474 v:vCMD
475 r@:{#vCMD# PATTERN <~ Display list of ONLY files matching regular expression PATTERN.}
476 | v:vMSG
477 j:lh1_present
478 l:lN_h22
479
480 f!:{^(del|delete|rm)$}:lN_h23
481 v:vCMD
482 r@:{#vCMD# PATTERN <~ Remove from storage, any and all files whose name matches PATTERN
    . CAN NOT DELETE registered user program files though.} | v:vMSG
483 j:lh1_present
484 l:lN_h23
485
486 f!:{^(rename|rn|mv|move)$}:lN_h24
487 v:vCMD
488 r@:{#vCMD# FILENAME NEWNAME <~ Change the name of an existing file from FILENAME to
    NEWNAME. WILL ALSO update the registered [user] programs list if FILENAME was a
    registered program. #vFILENAME_PROTOCOL#} | v:vMSG
489 j:lh1_present
490 l:lN_h24
491
492 f!:{^(exists)$}:lN_h25
493 v:vCMD
494 r@:{#vCMD# FILENAME <~ Show whether or not a file named FILENAME exists. #

```



```

    vFILENAME_PROTOCOL#} | v:vMSG
495 j:lh1_present
496 l:1N_h25
497
498 f!:{^(copy|clone)$}:1N_h26
499 v:vCMD
500 r@:{#vCMD# FILENAME FILE2 <~ Copies contents of FILENAME to FILE2 creating the
    destination if it doesn't exist. #vFILENAME_PROTOCOL#} | v:vMSG
501 j:lh1_present
502 l:1N_h26
503
504 f!:{^(runnable|register|add-program)$}:1N_h27
505 v:vCMD
506 r@:{#vCMD# FILENAME <~ Recognize FILENAME object as an executable [user] program or
    command. FILENAME must already be an existing TEA file. #vFILENAME_PROTOCOL#} | v:
    vMSG
507 j:lh1_present
508 l:1N_h27
509
510 f!:{^programs$}:1N_h28
511 v:vCMD
512 r@:{#vCMD# <~ Show list of all available registered [TEA] user programs by their
    registered names.
513
514 #vCMD# PATTERN <~ With PATTERN specified, ONLY show those programs whose names match
    regular expression PATTERN.}
515 | v:vMSG
516 j:lh1_present
517 l:1N_h28
518
519 f!:{^(run|exec|execute)$}:1N_h29
520 v:vCMD
521 r@:{#vCMD# FILENAME <~ Execute the given file record as a [TEA] program. #
    vFILENAME_PROTOCOL#} | v:vMSG
522 j:lh1_present
523 l:1N_h29
524
525 f!:{^(isrunnable|canrun)$}:1N_h30
526 v:vCMD
527 r@:{#vCMD# FILENAME <~ Show whether or not a file named FILENAME can be executed as a [
    TEA] program. #vFILENAME_PROTOCOL#} | v:vMSG
528 j:lh1_present
529 l:1N_h30
530
531 f!:{^(del-prog|delete-program|unregister)$}:1N_h31
532 v:vCMD
533 r@:{#vCMD# FILENAME <~ Revokes status of FILENAME as an executable user program (doesn'
    t actually delete the underlying file object though). #vFILENAME_PROTOCOL#} | v:vMSG
534 j:lh1_present
535 l:1N_h31
536
537
538 f!:{^(update|update-program)$}:1N_h32
539 v:vCMD
540 r@:{#vCMD# PROGRAMFILE FILENAME <~ Replace/override original program named PROGRAMFILE
    with contents of FILENAME (updates a program's code [in PROGRAMFILE] preserving its
    existing name). Both files PROGRAMFILE and FILENAME must actually exist for this to
    work, but only PROGRAMFILE needs to be a registered program. #vFILENAME_PROTOCOL#} |
    v:vMSG
541 j:lh1_present
542 l:1N_h32
543
544 f!:{^(program)$}:1N_h33
545 v:vCMD | z!: | v:vCMD #use upcase..
546 r@:{#vCMD# <~ Executes #vCMD# as a system command if it exists --- first, if it is any
    of the standard commands in TEA OS, otherwise if it is a valid filename "#vCMD#"

```

```

        that is already recognized/registered as a user-created program - this later case
        similar to calling "run #vCMD#", otherwise shows error message command-not-found.} |
        v:vMSG
547 j:lh1_present
548 l:1N_h33
549
550 # the user-program catch-all...
551 f!:~([a-z0-9_]+(\.[a-z0-9]+)?):1N_h34
552 v:vPROGRAMFILE
553 y@:dfProgramExists | v:fProgramExists #import
554 y:vPROGRAMFILE | e*:fProgramExists
555 f!:~PROGRAM EXISTS$:1N_h34
556 r@:{#vPROGRAMFILE# <~ Is a valid user-registered [TEA] program. You might run or
        execute it by directly invoking "#vPROGRAMFILE#" or via "run #vPROGRAMFILE#".} | v:
        vMSG
557 j:lh1_present
558 l:1N_h34
559
560 l:lh1_present
561 y@:dvDELIM2|v:vD
562 g*:{}:vD:vMSG:vD
563 "
564 L:1SYS_HELP_END
565 L:1SYS_FILE_START
566 V:fGetFilenameProtocolMSG:"i!:{Valid FILENAME should consist of ONLY letters a-zA-Z,
        numbers 0-9, underscore _ and/or a dot .}"
567 Y:fGetFilenameProtocolMSG
568 V@:dfGetFilenameProtocolMSG
569 V:fGenDBFilename:"x:{DFS_}"
570 Y:fGenDBFilename
571 V@:dfGenDBFilename
572 V:fDecodeDBFilename:"d:~DFS_"
573 Y:fDecodeDBFilename
574 V@:dfDecodeDBFilename
575 V:fIsFileNameValid:"
576 f!:^[a-z0-9_]+(\.[a-z0-9]+)?$:1BAD_FILE_NAME
577 i!:1|q!:
578 l:1BAD_FILE_NAME
579 i!:0
580 "
581 Y:fIsFileNameValid
582 V@:dfIsFileNameValid
583 V:fCreateFile:"
584 # first, grab incoming function argument...
585 v:vFILENAME|t!.:|v:vFNAME
586
587 #then import functions..
588 y@:dfGenDBFilename|v:fGenDBFilename
589 y@:dfGetLatestOutputBuffer|v:fGetLatestOutputBuffer
590 y@:dfIsFileNameValid|v:fIsFileNameValid
591
592 y:vFNAME | f:~$:1NFINE # filename shouldn't be blank
593 # file name should conform to accepted patterns
594 y:vFNAME | E*:fIsFileNameValid | f!:0:1GOOD_FILENAME
595 r@:{Invalid File Name [#vFNAME#]!
596 FILE NOT CREATED.}|q!:
597 l:1GOOD_FILENAME
598
599 # fetch current file name list..
600 y@:dvFileList | v:vFileList
601 # ensure file name doesn't yet exist in records
602 #e.g v:vFileList:{F1|F2|F33|File|File}
603 #v:vFNAME:{f1}
604 # check if file name is unique
605 r@:{(~#vFNAME#\)|(\|#vFNAME#\)|(\|#vFNAME#$)}|v:vFNAME_REGEX
606 y:vFileList | f*!:vFNAME_REGEX:1FNAME_UNIQUE:1FNAME_DUPLICATE

```

```

607 l:lFNAME_UNIQUE
608 # add name to file list
609 g*:{|}:vFileList:vFNAME | v:vFileList | v@:dvFileList
610 #also create blank file record in db filesystem
611 y:vFNAME|E*:fGenDBFilename|v:vdFNAME
612
613 v@:dvDATA:{} # we'll write empty file
614 r@:{y@:dvDATA|v@:#vdFNAME#}|E:
615
616 i!:{CREATED NEW BLANK FILE: }|x*!:vFNAME
617 q!:
618 l:lFNAME_DUPLICATE
619 i!:{File Already Exists!
620 NOT DUPLICATING FILE: }|x*!:vFNAME
621 q!:
622 l:lNFINE
623 i!:{Invalid [blank] File Name!
624 FILE NOT CREATED.}
625 "
626 Y:fCreateFile
627 V@:dfCreateFile
628 V:fDeleteFile:"
629 # first, grab incoming function argument...
630 v:vFILENAME|t!.:|v:vFNAME
631
632 #then import functions..
633 y@:dfGenDBFilename|v:fGenDBFilename
634 y@:dfIsFileNameValid|v:fIsFileNameValid
635
636 y:vFNAME | f:~$:lNFINE # filename shouldn't be blank
637 # file name should conform to accepted patterns
638 y:vFNAME | E*:fIsFileNameValid | f!:0:lGOOD_FILENAME
639 r@:{Invalid File Name [#vFNAME#]!
640 FILE NOT DELETED.}|q!:
641 l:lGOOD_FILENAME
642
643 # fetch current file name list..
644 y@:dvFileList | v:vFileList
645 # remove file name from records
646
647 #v:vFileList:{F1|F2|F33|F3|File|File}
648 #v:vFNAME:{File}
649 # get filename pattern
650
651 # create delete filename patterns + apply them..
652 r@:{(^#vFNAME#\\)|\\(#vFNAME#$)}|v:vFNAME_REGEX_ENDS
653 r@:{\\(#vFNAME#\\)|}\\v:vFNAME_REGEX_MID
654 r@:{^#vFNAME#$}|v:vFNAME_REGEX_ALL
655 # use it to del
656 y:vFileList | d*:vFNAME_REGEX_ENDS | v:vFileList
657 y:vFileList | d*:vFNAME_REGEX_ALL | v:vFileList
658 v:vDIV:{}
659 r*:vFileList:vFNAME_REGEX_MID:vDIV | v:vFileList
660
661 # update file list
662 y:vFileList | v@:dvFileList
663 #also delete file record from db filesystem
664 y:vFNAME|E*:fGenDBFilename|v:vdFNAME
665 r@:{c!@:#vdFNAME#}|E:
666
667 i!:{DELETED FILE: }|x*!:vFNAME
668 q!:
669
670 l:lNFINE
671 i!:{Invalid [blank] File Name!
672 FILE NOT DELETED.}

```

```

673 "
674 Y:fDeleteFile
675 V:dfDeleteFile
676 V:fShowFilesList:"
677 # takes no parameters ;)
678 # fetch current file name list..
679 y@:dvFileList
680 #process and present..
681 h!:{\|}|d:{\|}
682 "
683 V:fSearchFilesList:"
684 # takes file regex pattern parameter
685 v:vREGEX_PATTERN
686 # fetch current file name list..
687 y@:dvFileList
688 #process and present..
689 h!:{\|}|d:{\|}
690 # only keep what matches...
691 k*:vREGEX_PATTERN
692 "
693 V:fDeleteFilesList:"
694 # takes file regex pattern parameter
695 v:vREGEX_PATTERN
696
697 #imports..
698 y@:dfDeleteFile | v:fDeleteFile
699 y@:dfProgramExists | v:fProgramExists
700
701 # fetch current file name list..
702 y@:dvFileList
703 #process and present..
704 h!:{\|}|d:{\|}
705 # only keep what matches...
706 k*:vREGEX_PATTERN
707 g:{|} | v:vDelFileList | v:vDelFileListORIG
708 f:^$:NO_FILES_TO_DELETE
709 # to track files we couldn't delete
710 v:vNOTDelFileList:{}
711
712 # go through files and delete..
713 v:vNCOUNTER:0
714 v:vNCOUNTER_DEL:0
715
716 l:lSTART_DEL
717 y:vDelFileList
718 f:^$:lDONE_DEL
719 #fetch..
720 y:vDelFileList | d!:{^[^\|]+\|?}
721 d:{\|} | v:vDelFileName
722
723 #if actually blank.. go pop
724 f:^$:lPOPDEL_LIST
725
726 # ensure it is not a registered program
727 y:vDelFileName | e*:fProgramExists | f:{^PROGRAM EXISTS$}:lDELETE_FAILED
728
729 #r@:{--Continuing to delete file [#vDelFileName#]} | q!:
730
731 # delete file..
732 y:vDelFileName | e*:fDeleteFile
733 #q!: # check...
734
735 f!:{DELETED FILE}:lDELETE_FAILED:lDELETED_OK
736 l:lDELETE_FAILED
737 # add name to NOT deleted file list
738 g*:{|}:vNOTDelFileList:vDelFileName | v:vNOTDelFileList

```

```

739 j:lDONE_DELETED_OK
740 l:lDELETED_OK
741 y:vNCOUNTER_DEL | x!:+1 | r.: | v:vNCOUNTER_DEL
742 l:lDONE_DELETED_OK
743 y:vNCOUNTER | x!:+1 | r.: | v:vNCOUNTER
744
745 #pop..
746 l:lPOPDEL_LIST
747 y:vDelFileList | d:{^[^\|]+\|?} | v:vDelFileList
748 #x:{Remaining: } | q!:
749 j:lSTART_DEL # iterate
750
751 l:lDONE_DEL
752 y:vDelFileListORIG | d:{^[^\|}:{\|}$}| v:vDelFileListORIG
753 y:vNOTDelFileList | d:{^[^\|}:{\|}$}| v:vNOTDelFileList
754 r@:{#vNCOUNTER# - #vNCOUNTER_DEL#}|r.:|v:vNCOUNTER_NDEL
755 y:vNCOUNTER_DEL | f:~1$:lDEL_REP_1
756 r@:{DELETED #vNCOUNTER_DEL# file[s] from list:
757 #vDelFileListORIG#
758 EXCEPT #vNCOUNTER_NDEL#:
759 #vNOTDelFileList#}|q!:
760
761 l:lDEL_REP_1
762 y:vDelFileListORIG | d:{^[^\|}:{\|}$}| v:vDelFileListORIG
763 y:vNCOUNTER | f!:~1$:lDEL_REP_2
764 r@:{DELETED ONLY #vNCOUNTER_DEL# file:
765 #vDelFileListORIG#
766 }|q!:
767
768 l:lDEL_REP_2
769 y:vDelFileListORIG | d:{^[^\|}:{\|}$}| v:vDelFileListORIG
770 y:vNOTDelFileList | d:{^[^\|}:{\|}$}| v:vNOTDelFileList
771 r@:{DELETED ONLY #vNCOUNTER_DEL# file from list:
772 #vDelFileListORIG#
773 EXCEPT #vNCOUNTER_NDEL#:
774 #vNOTDelFileList#}|q!:
775
776 l:NO_FILES_TO_DELETE
777 r@:{#vREGEX_PATTERN# matched no FILES!
778 NO FILE WAS DELETED}
779 "
780 V:fInitFile:"
781 v:vFILENAME|t!.:|v:vFNAME
782
783 #imports...
784 y@:dfCreateFile|v:fCreateFile
785 y@:dfGenDBFilename|v:fGenDBFilename
786 y@:dfGetLatestOutputBuffer|v:fGetLatestOutputBuffer
787 y@:dfIsFileNameValid|v:fIsFileNameValid
788
789 y:vFNAME | E*:fIsFileNameValid | f!:0:lGOOD_FILENAME
790 r@:{Invalid File Name [#vFNAME#]!
791 FILE NOT CREATED.}|q!:
792 l:lGOOD_FILENAME
793
794 # then, try to create the file...
795 y:vFNAME | E*:fCreateFile
796 # check if it was created...
797 f!:{CREATED NEW BLANK FILE}:lFILE_NOT_CREATED
798 # get name of that newly created file...
799 y:vFNAME|E*:fGenDBFilename|v:vdFNAME
800
801 # get current output buffer
802 y@:dvATABUFFER | v@:dvDATA
803
804 # only if it was empty then...

```

```

805 f!:~$:lFINE_DATA_BUFFER
806 # use print buffer...
807 e*:fGetLatestOutputBuffer | v@:dvDATA
808 l:lFINE_DATA_BUFFER
809
810 # store it into file system record...
811 r@:{y@:dvDATA|v@:#vdFNAME#}|E:
812
813 #a test...
814 #r@:{y@:#vdFNAME#}|E:
815 #x:{File now contains:
816 #}|q!:
817
818 i!:{CREATED and INITIALIZED NEW File: }|x*!:vFNAME
819 q!:
820 l:lFILE_NOT_CREATED
821 i!:{FAILED TO CREATE FILE: } | x*!:vFNAME
822 "
823 V:fWriteFile:"
824 v:vFILENAME|t!..|v:vFNAME
825
826 #imports...
827 y@:dfCreateFile|v:fCreateFile
828 y@:dfGenDBFilename|v:fGenDBFilename
829 y@:dfGetLatestOutputBuffer|v:fGetLatestOutputBuffer
830 y@:dfIsFileNameValid|v:fIsFileNameValid
831
832 y:vFNAME | E*:fIsFileNameValid | f!:0:lGOOD_FILENAME
833 r@:{Invalid File Name [#vFNAME#]!
834 FILE NOT WRITTEN.}|q!:
835 l:lGOOD_FILENAME
836
837 # then, try to create the file...
838 y:vFNAME | E*:fCreateFile
839 # check if it was created...
840 f!:{(CREATED NEW BLANK FILE)|(NOT DUPLICATING FILE)}:lFILE_NOT_CREATED
841 # get name of that file...
842 y:vFNAME|E*:fGenDBFilename|v:vdFNAME
843 # get current output buffer
844 y@:dvDATABUFFER | v@:dvDATA
845
846 # only if it was empty then...
847 f!:~$:lFINE_DATA_BUFFER
848 # use print buffer...
849 e*:fGetLatestOutputBuffer | v@:dvDATA
850 l:lFINE_DATA_BUFFER
851
852 # store it into file system record...
853 r@:{y@:dvDATA|v@:#vdFNAME#}|E:
854
855 i!:{DATA WRITTEN to File: }|x*!:vFNAME
856 q!:
857 l:lFILE_NOT_CREATED
858 i!:{FAILED TO WRITE FILE: } | x*!:vFNAME
859 "
860 V:fSaveDataToFile:"
861 v:vFILENAME|t!..|v:vFNAME
862
863 #imports...
864 y@:dfCreateFile|v:fCreateFile
865 y@:dfGenDBFilename|v:fGenDBFilename
866 y@:dfGetLatestOutputBuffer|v:fGetLatestOutputBuffer
867 y@:dfIsFileNameValid|v:fIsFileNameValid
868
869 y:vFNAME | E*:fIsFileNameValid | f!:0:lGOOD_FILENAME
870 r@:{Invalid File Name [#vFNAME#]!

```

```

871 FILE NOT WRITTEN.}|q!:
872 l:lGOOD_FILENAME
873
874 # then, try to create the file...
875 y:vFNAME | E*:fCreateFile
876 # check if it was created...
877 f!:{(CREATED NEW BLANK FILE)|(NOT DUPLICATING FILE)}:lFILE_NOT_CREATED
878 # get name of that file...
879 y:vFNAME|E*:fGenDBFilename|v:vdFNAME
880 # get current data buffer
881 # and store it into file system record...
882 r@:{y@:dvDATABUFFER|v@:#vdFNAME#}|E:
883
884 i!:{DATA WRITTEN to File: }|x*!:vFNAME
885 q!:
886 l:lFILE_NOT_CREATED
887 i!:{FAILED TO WRITE FILE: } | x*!:vFNAME
888 "
889 V:fReadFile:"
890 v:vFILENAME|t!.:|v:vFNAME
891
892 #imports...
893 y@:dfGenDBFilename|v:fGenDBFilename
894 y@:dfIsFileNameValid|v:fIsFileNameValid
895
896 y:vFNAME | E*:fIsFileNameValid | f!:0:lGOOD_FILENAME
897 r@:{Invalid File Name [#vFNAME#]}!
898 FILE NOT READ.}|q!:
899 l:lGOOD_FILENAME
900
901 # then, try to read the file...
902 y:vFNAME|E*:fGenDBFilename|v:vdFNAME
903
904 r@:{y@:#vdFNAME#}|E:
905 "
906 y:fReadFile
907 v@:dfReadFile
908 V:fFileExists:"
909 # first, grab incoming function argument...
910 v:vFILENAME|t!.:|v:vFNAME
911
912 #then import functions..
913 y@:dfGenDBFilename|v:fGenDBFilename
914 y@:dfIsFileNameValid|v:fIsFileNameValid
915
916 y:vFNAME | f:~$:lNFINE
917 y:vFNAME | E*:fIsFileNameValid | f!:0:lGOOD_FILENAME
918 r@:{Invalid File Name [#vFNAME#]}!
919 FILE MAY NOT EXIST.}|q!:
920 l:lGOOD_FILENAME
921
922 # fetch current file name list..
923 y@:dvFileList | v:vFileList
924
925
926 #v:vFileList:{F1|F2|F33|F3|File|File}
927 #v:vFNAME:{f1}
928
929 # create filename patterns..
930 r@:{(^#vFNAME$|)|(\|#vFNAME$)}|v:vFNAME_REGEX_ENDS
931 r@:{(\|#vFNAME$|)|}|v:vFNAME_REGEX_MID
932 r@:{^#vFNAME#$}|v:vFNAME_REGEX_ALL
933 # use them to test for existence
934 y:vFileList | f*:vFNAME_REGEX_ENDS:lFILE_EXISTS
935 f*:vFNAME_REGEX_ALL:lFILE_EXISTS
936 f*:vFNAME_REGEX_MID:lFILE_EXISTS

```

```

937
938 i!:{FILE DOES NOT EXIST}|q!:
939
940 l:1FILE_EXISTS
941 i!:{FILE EXISTS}
942 q!:
943
944 l:1NFINE
945 i!:{Invalid [blank] File Name!
946 FILE DOES NOT EXIST.}
947 "
948 L:1SYS_FILE_END
949 L:1SYS_PROG_START
950 V:fRegisterProgramFile:"
951 # we assume given program file already exists!
952 # first, grab incoming function argument...
953 v:vFILENAME|t!.:|v:vFNAME
954
955 #then import functions..
956 y@:dfGenDBFilename|v:fGenDBFilename
957 y@:dfIsFileNameValid|v:fIsFileNameValid
958
959 y:vFNAME | f:^$:1NFINE
960 y:vFNAME | E*:fIsFileNameValid | f!:0:1GOOD_FILENAME
961 r@:{Invalid File Name [#vFNAME#]!
962 CANNOT REGISTER PROGRAM.}|q!:
963 l:1GOOD_FILENAME
964
965 # fetch current program file name list..
966 y@:dvProgramsList | v:vProgramsList
967
968 #v:vFileList:{F1|F2|F33|F3|File|File}
969 #v:vFNAME:{f1}
970
971 # check if program name is unique:
972 # create filename patterns...
973 r@:{(^#vFNAME#|)|(\|#vFNAME#|)|(\|#vFNAME#$)}|v:vFNAME_REGEX
974 # use them to test for program existence
975 y:vProgramsList | f*!:vFNAME_REGEX:1PROGRAM_NAME_UNIQUE:1PROG_EXISTS
976 l:1PROGRAM_NAME_UNIQUE
977 # So, program isn't registered yet, meaning its ok
978 # to proceed to register it...
979 # add name to programs list
980 g*:{|}:vProgramsList:vFNAME | v:vProgramsList | v@:dvProgramsList
981 r@:{User PROGRAM [#vFNAME#] is now REGISTERED.
982 CAN BE INVOKED as SYSTEM COMMAND.} | q!:
983
984 l:1PROG_EXISTS
985 r@:{User PROGRAM [#vFNAME#] Already EXISTS!
986 CANNOT re-REGISTER PROGRAM.} | q!:
987
988 l:1NFINE
989 i!:{Invalid [blank] File Name!
990 CANNOT REGISTER PROGRAM.}
991 "
992 V:fShowProgramsList:"
993 y@:dvProgramsList|h!:{\}|d:{\}|
994 "
995 V:fSearchProgramsList:"
996 v:vREGEX_PATTERN
997 y@:dvProgramsList|h!:{\}|d:{\}|k*:vREGEX_PATTERN
998 "
999 V:fRunFile:"
1000 v:vFILENAME|t!.:|v:vFNAME
1001
1002 #imports...

```



```

1003 y@:dfGenDBFilename|v:fGenDBFilename
1004 y@:dfIsFileNameValid|v:fIsFileNameValid
1005
1006 y:vFNAME | E*:fIsFileNameValid | f!:0:lGOOD_FILENAME
1007 r@:{Invalid File Name [#vFNAME#]!
1008 FILE NOT EXECUTED.}|q!:
1009 l:lGOOD_FILENAME
1010
1011 # then, try to run the file...
1012 y:vFNAME|E*:fGenDBFilename|v:vdFNAME
1013
1014 # for now, run it as a TEA program..
1015 r@:{y@:#vdFNAME#}|E:|E:
1016 "
1017 V:fProgramExists:"
1018 v:vFILENAME|t!.:|v:vFNAME
1019
1020 y@:dfGenDBFilename|v:fGenDBFilename
1021 y@:dfIsFileNameValid|v:fIsFileNameValid
1022
1023 y:vFNAME | f:~$:lNFINE
1024 y:vFNAME | E*:fIsFileNameValid | f!:0:lGOOD_FILENAME
1025 r@:{Invalid File Name [#vFNAME#]!
1026 PROGRAM MAY NOT EXIST.}|q!:
1027 l:lGOOD_FILENAME
1028
1029 # fetch current programs name list..
1030 y@:dvProgramsList | v:vProgramsList
1031
1032 r@:{(^#vFNAME#\)|(\|#vFNAME#\)|(\|#vFNAME#$))|v:vFNAME_REGEX
1033 y:vProgramsList | f*:vFNAME_REGEX:lPROG_EXISTS
1034
1035 i!:{PROGRAM DOES NOT EXIST}|q!:
1036
1037 l:lPROG_EXISTS
1038 i!:{PROGRAM EXISTS}|q!:
1039
1040 l:lNFINE
1041 i!:{Invalid [blank] File Name!
1042 PROGRAM DOES NOT EXIST.}
1043 "
1044 Y:fProgramExists
1045 V@:dfProgramExists
1046 V:fUnRegisterProgram:"
1047 # we assume given program file already exists!
1048 # first, grab incoming function argument...
1049 v:vFILENAME|t!.:|v:vFNAME
1050
1051 #then import functions..
1052 y@:dfGenDBFilename|v:fGenDBFilename
1053 y@:dfIsFileNameValid|v:fIsFileNameValid
1054
1055 y:vFNAME | f:~$:lNFINE
1056 y:vFNAME | E*:fIsFileNameValid | f!:0:lGOOD_FILENAME
1057 r@:{Invalid File Name [#vFNAME#]!
1058 CANNOT UNREGISTER PROGRAM.}|q!:
1059 l:lGOOD_FILENAME
1060
1061 # fetch current program file name list..
1062 y@:dvProgramsList | v:vProgramsList
1063
1064 # create delete filename patterns + apply them..
1065 r@:{(^#vFNAME#\)|(\|#vFNAME#$))|v:vFNAME_REGEX_ENDS
1066 r@:{(\|#vFNAME#\)|}|v:vFNAME_REGEX_MID
1067 r@:{^#vFNAME#$)|v:vFNAME_REGEX_ALL
1068 # use it to del

```

```

1069 y:vProgramsList | d*:vFNAME_REGEX_ENDS | v:vProgramsList
1070 y:vProgramsList | d*:vFNAME_REGEX_ALL | v:vProgramsList
1071 v:vDIV:{|}
1072 r*:vProgramsList:vFNAME_REGEX_MID:vDIV | v:vProgramsList
1073
1074 # update programs list
1075 y:vProgramsList | v@:dvProgramsList
1076
1077 i!:{UNREGISTERED PROGRAM FILE: }|x*!:vFNAME
1078 q!:
1079
1080 l:1NFINE
1081 i!:{Invalid [blank] File Name!
1082 PROGRAM FILE NOT UNREGISTERED.}
1083 "
1084 Y:fUnRegisterProgram
1085 V@:dfUnRegisterProgram
1086 V:fDeleteProgramFile:"
1087 # first, grab incoming function argument...
1088 v:vFILENAME|t!.:|v:vFNAME
1089
1090 #then import functions..
1091 y@:dfGenDBFilename|v:fGenDBFilename
1092 y@:dfIsFileNameValid|v:fIsFileNameValid
1093 y@:dfUnRegisterProgram |v:fUnRegisterProgram
1094
1095 y:vFNAME | f:~$:1NFINE # filename shouldn't be blank
1096 # file name should conform to accepted patterns
1097 y:vFNAME | E*:fIsFileNameValid | f!:0:1GOOD_FILENAME
1098 r@:{Invalid File Name [#vFNAME#]!
1099 PROGRAM FILE NOT DELETED.}|q!:
1100 l:1GOOD_FILENAME
1101
1102 # fetch current file name list..
1103 y@:dvFileList | v:vFileList
1104
1105
1106 # create delete filename patterns + apply them..
1107 r@:{(~#vFNAME#)|(\|#vFNAME#)}|v:vFNAME_REGEX_ENDS
1108 r@:{(\|#vFNAME#)|}|v:vFNAME_REGEX_MID
1109 r@:{~#vFNAME#}$|v:vFNAME_REGEX_ALL
1110 # use it to del
1111 y:vFileList | d*:vFNAME_REGEX_ENDS | v:vFileList
1112 y:vFileList | d*:vFNAME_REGEX_ALL | v:vFileList
1113 v:vDIV:{|}
1114 r*:vFileList:vFNAME_REGEX_MID:vDIV | v:vFileList
1115
1116 # update file list
1117 y:vFileList | v@:dvFileList
1118 #also delete file record from db filesystem
1119 y:vFNAME|e*:fGenDBFilename|v:vdFNAME
1120 r@:{c!@:#vdFNAME#}|E:
1121
1122 # then unregister program file
1123 y:vFNAME | e*:fUnRegisterProgram | v:vUnRegStatus
1124
1125 r@:{DELETED PROGRAM FILE: #vFNAME#
1126 With Status: #vUnRegStatus#} | q!:
1127
1128
1129 l:1NFINE
1130 i!:{Invalid [blank] File Name!
1131 PROGRAM FILE NOT DELETED.}
1132 "
1133 Y:fDeleteProgramFile
1134 V@:dfDeleteProgramFile

```

```

1135 L:1SYS_PROG_END
1136 L:1SYS_SHELL_START
1137 E*:fLogin
1138 L:1START_Main
1139 E*:fMakePS
1140 V@:dvLastPS
1141 E*:fPROMPT
1142 E*:fLogAI
1143 V:vIN_raw
1144 t!.:
1145 v:vIN_clean
1146 Z:
1147 v:vIN_clean_lc
1148 y:vIN_clean_lc
1149 F!:^create\s+.*$:1N_CRT_FILE
1150 Y:vIN_raw
1151 t!.:
1152 V@:dvLastCommand
1153 d:^\S+\s+
1154 V:vFileName
1155 E*:fCreateFile
1156 E*:fLog0
1157 J:1FIN_CMD_PROCESSING
1158 L:1N_CRT_FILE
1159 F!:^(ls|list|dir|files)$:1N_LIST_FILES
1160 V@:dvLastCommand
1161 E*:fShowFilesList
1162 E*:fLog0
1163 J:1FIN_CMD_PROCESSING
1164 L:1N_LIST_FILES
1165 F!:^(ls|list|dir|files|find)\s+(\S+\s*)+$:1N_SEARCH_FILES
1166 Y:vIN_raw
1167 t!.:
1168 V@:dvLastCommand
1169 d:^\S+\s+
1170 V:vFilePattern
1171 E*:fSearchFilesList
1172 E*:fLog0
1173 J:1FIN_CMD_PROCESSING
1174 L:1N_SEARCH_FILES
1175 F!:^(del|delete|rm)\s+(\S+\s*)+$:1N_DEL_FILES
1176 Y:vIN_raw
1177 t!.:
1178 V@:dvLastCommand
1179 d:^\S+\s+
1180 V:vFilePattern
1181 E*:fDeleteFilesList
1182 E*:fLog0
1183 J:1FIN_CMD_PROCESSING
1184 L:1N_DEL_FILES
1185 F!:^init\s+.*$:1N_INIT_FILE
1186 Y:vIN_raw
1187 t!.:
1188 V@:dvLastCommand
1189 d:^\S+\s+
1190 V:vFileName
1191 E*:fInitFile
1192 E*:fLog0
1193 J:1FIN_CMD_PROCESSING
1194 L:1N_INIT_FILE
1195 F!:^write\s+.*$:1N_WRITE_FILE
1196 Y:vIN_raw
1197 t!.:
1198 V@:dvLastCommand
1199 d:^\S+\s+
1200 V:vFileName

```

```

1201 E*:fWriteFile
1202 E*:fLog0
1203 J:lFIN_CMD_PROCESSING
1204 L:lN_WRITE_FILE
1205 F!:^(show|cat|read)\s+.*$}:lN_READ_FILE
1206 Y:vIN_raw
1207 t!.:
1208 V@:dvLastCommand
1209 d:^\S+\s+
1210 V:vFileName
1211 E*:fReadFile
1212 E*:fLog0
1213 J:lFIN_CMD_PROCESSING
1214 L:lN_READ_FILE
1215 F!:^(exists)\s+.*$}:lN_EXISTS_FILE
1216 Y:vIN_raw
1217 t!.:
1218 V@:dvLastCommand
1219 d:^\S+\s+
1220 V:vFileName
1221 E*:fFileExists
1222 E*:fLog0
1223 J:lFIN_CMD_PROCESSING
1224 L:lN_EXISTS_FILE
1225 F!:^(rename|ln|mv|move)\s+([a-z0-9_]+(\.[a-z0-9]+)?)\s+([a-z0-9_]+(\.[a-z0-9]+)?)$}:
    lN_RENAME_FILE
1226 Y:vIN_raw
1227 t!.:
1228 V@:dvLastCommand
1229 d:^\S+\s+
1230 V:vFileNames
1231 Y:vFileNames
1232 D:\s+.*$
1233 V:vFileName
1234 Y:vFileNames
1235 D!:\s+.*$
1236 V:vNewName
1237 Y:vFileName
1238 E*:fFileExists
1239 F!:^FILE EXISTS$}:lCANNOT_RENAME
1240 Y:vFileName
1241 E*:fProgramExists
1242 F:{^PROGRAM EXISTS$}:lPROCESS_RENAME_PROGRAM
1243 Y:vFileName
1244 E*:fReadFile
1245 v@:dvDATABUFFER
1246 Y:vNewName
1247 E*:fSaveDataToFile
1248 v:vWRITELOG
1249 Y:vFileName
1250 E*:fDeleteFile
1251 R@:{File #vFileName# RENAMED #vNewName#
1252 With status: #vWRITELOG#}
1253 E*:fLog0
1254 J:lFIN_CMD_PROCESSING
1255 L:lPROCESS_RENAME_PROGRAM
1256 Y:vFileName
1257 E*:fReadFile
1258 v@:dvDATABUFFER
1259 Y:vNewName
1260 E*:fSaveDataToFile
1261 v:vWRITELOG
1262 Y:vNewName
1263 E*:fRegisterProgramFile
1264 v:vREGLOG
1265 Y:vFileName

```

```

1266 E*:fDeleteProgramFile
1267 v:vDELLOG
1268 R@:{Program File #vFileName# RENAMED #vNewName#
1269 With [WRITE] status: #vWRITELOG#
1270 And [REGISTER] status: #vREGLOG#
1271 And [DEL] status: #vDELLOG#}
1272 E*:fLog0
1273 J:lFIN_CMD_PROCESSING
1274 L:lCANNOT_RENAME
1275 R@:{File #vFileName# does NOT EXIST!
1276 RENAME Operation ABANDONED}
1277 E*:fLog0
1278 J:lFIN_CMD_PROCESSING
1279 L:lN_RENAME_FILE
1280 F!:{^(copy|clone)\s+([a-z0-9_]+(\.[a-z0-9]+)?)\s+([a-z0-9_]+(\.[a-z0-9]+)?)$}:
    lN_CLONE_FILE
1281 Y:vIN_raw
1282 t!.:
1283 V@:dvLastCommand
1284 d:~\S+\s+
1285 V:vFileNames
1286 Y:vFileNames
1287 D:\s+.*$
1288 V:vFileName
1289 Y:vFileNames
1290 D!:\s+.*$
1291 V:vFileName2
1292 Y:vFileName
1293 E*:fFileExists
1294 F!:{^FILE EXISTS$}:lCANNOT_CLONE
1295 Y:vFileName
1296 E*:fReadFile
1297 v@:dvDATABUFFER
1298 Y:vFileName2
1299 E*:fSaveDataToFile
1300 v:vWRITELOG
1301 R@:{File #vFileName# CLONED TO #vFileName2#
1302 With status: #vWRITELOG#}
1303 E*:fLog0
1304 J:lFIN_CMD_PROCESSING
1305 l:lCANNOT_CLONE
1306 R@:{File #vFileName# does NOT EXIST!
1307 CLONE Operation ABANDONED}
1308 E*:fLog0
1309 J:lFIN_CMD_PROCESSING
1310 L:lN_CLONE_FILE
1311 F!:{^export\s+.*$:lN_EXPORT_FILE
1312 Y:vIN_raw
1313 t!.:
1314 V@:dvLastCommand
1315 d:~\S+\s+
1316 V:vFileName
1317 E*:fReadFile
1318 v:vLOG
1319 E*:fResetLog
1320 E*:fOSHandle
1321 X:{Thanks for Using }
1322 V:vMSG
1323 G*!:vDELIM2:vFileName:vLOG:vMSG
1324 J:lEXIT_Main
1325 L:lN_EXPORT_FILE
1326 F!:{^uimini$:lN_UIMINI
1327 V@:dvLastCommand
1328 V@:dvUIMODE_MINI:{1}
1329 I!:{TOS Mini-Interface Mode Activated}
1330 E*:fLog0

```

```

1331 J:1FIN_CMD_PROCESSING
1332 L:1N_UIMINI
1333 F!:~uilog$:1N_UILOG
1334 V@:dvLastCommand
1335 V@:dvUIMODE_MINI:{0}
1336 I!:{TOS Log-Interface Mode Activated}
1337 E*:fLog0
1338 J:1FIN_CMD_PROCESSING
1339 L:1N_UILOG
1340 F!:~about$:1N_ABOUT
1341 V@:dvLastCommand
1342 E*:fGetAboutTOS
1343 E*:fLog0
1344 J:1FIN_CMD_PROCESSING
1345 L:1N_ABOUT
1346 F!:{^(help|man|whatis|info)$}:1N_HELP
1347 V@:dvLastCommand
1348 E*:fGetHelpMenu
1349 E*:fLog0
1350 J:1FIN_CMD_PROCESSING
1351 L:1N_HELP
1352 F!:{^(help|man|whatis|info|about)\s+(\S+\s*)+$}:1N_HELP_TOPIC
1353 Y:vIN_raw
1354 t!.:
1355 V@:dvLastCommand
1356 d:{~\S+\s+}
1357 E*:fGetHelpOn
1358 E*:fLog0
1359 J:1FIN_CMD_PROCESSING
1360 L:1N_HELP_TOPIC
1361 F!:~help-file[s]?$:1N_HELP_FILE
1362 V@:dvLastCommand
1363 E*:fGetFileCMDMenu
1364 E*:fLog0
1365 J:1FIN_CMD_PROCESSING
1366 L:1N_HELP_FILE
1367 F!:{~help-prog([s]|rams)?$}:1N_HELP_PROG
1368 V@:dvLastCommand
1369 E*:fGetProgrammingMenu
1370 E*:fLog0
1371 J:1FIN_CMD_PROCESSING
1372 L:1N_HELP_PROG
1373 F!:~help-util[s]?$:1N_HELP_UT
1374 V@:dvLastCommand
1375 E*:fGetUtilityMenu
1376 E*:fLog0
1377 J:1FIN_CMD_PROCESSING
1378 L:1N_HELP_UT
1379 F!:~help-ui[s]?$:1N_HELP_UI
1380 V@:dvLastCommand
1381 E*:fGetUIMenu
1382 E*:fLog0
1383 J:1FIN_CMD_PROCESSING
1384 L:1N_HELP_UI
1385 F!:~help-help[s]?$:1N_HELP_HELP
1386 V@:dvLastCommand
1387 E*:fGetHelpMenu
1388 X:{
1389 To use or access help on any topic in the system, use any of the commands presented in
    the following Menu.
1390 }
1391 X!:{
1392 Each of those commands leads you to further information about help on other commands.
    For example
1393
1394 > help help-util

```

```

1395
1396 Will explain what the help-util command is all about.}
1397 E*:fLog0
1398 J:lFIN_CMD_PROCESSING
1399 L:lN_HELP_HELP
1400 F!:^echo.*$:lN_ECHO
1401 Y:vIN_raw
1402 t!.:
1403 d:^\S+\s*
1404 V:vMESSAGE
1405 X:{echo }
1406 V@:dvLastCommand
1407 Y:vMESSAGE
1408 E*:fLog0
1409 J:lFIN_CMD_PROCESSING
1410 L:lN_ECHO
1411 F!:{^(fetch|web)\s+\S*$}:lN_FETCH
1412 Y:vIN_raw
1413 t!.:
1414 V@:dvLastCommand
1415 d:{^\S+\s+}
1416 E*:fFixURL
1417 W:
1418 E*:fLog0
1419 J:lFIN_CMD_PROCESSING
1420 L:lN_FETCH
1421 F!:^tea\s+[\S]+.*$:lN_TEA
1422 Y:vIN_raw
1423 t!.:
1424 d:^\S+\s+
1425 V:vTEACode
1426 X:{tea }
1427 V@:dvLastCommand
1428 Y:vTEACode
1429 E:
1430 E*:fLog0
1431 J:lFIN_CMD_PROCESSING
1432 L:lN_TEA
1433 F!:^math\s+[\S]+.*$:lN_MATH
1434 Y:vIN_raw
1435 t!.:
1436 d:^\S+\s+
1437 V:vMathExp
1438 X:{math }
1439 V@:dvLastCommand
1440 Y:vMathExp
1441 R.:
1442 E*:fLog0
1443 J:lFIN_CMD_PROCESSING
1444 L:lN_MATH
1445 F!:^username\s+[\S]+.*$:lN_SETUSER
1446 V@:dvLastCommand
1447 Y:vIN_raw
1448 t!.:
1449 d:^\S+\s+
1450 V@:dvUSERNAME
1451 E*:fLog0
1452 J:lFIN_CMD_PROCESSING
1453 L:lN_SETUSER
1454 F!:{^(whoami|name)$}:lN_WHOAMI
1455 V@:dvLastCommand
1456 Y@:dvUSERNAME
1457 E*:fLog0
1458 J:lFIN_CMD_PROCESSING
1459 L:lN_WHOAMI
1460 F!:^time$:lN_TIME

```

```
1461 v@:dvLastCommand
1462 Z.:TIME
1463 E*:fLog0
1464 J:lFIN_CMD_PROCESSING
1465 L:lN_TIME
1466 F!:^date$:lN_DATE
1467 v@:dvLastCommand
1468 Z.:DATE
1469 E*:fLog0
1470 J:lFIN_CMD_PROCESSING
1471 L:lN_DATE
1472 F!:^timestamp$:lN_TSTAMP
1473 v@:dvLastCommand
1474 Z.:TIMESTAMP
1475 E*:fLog0
1476 J:lFIN_CMD_PROCESSING
1477 L:lN_TSTAMP
1478 F!:^{(clear|reset)}$:lN_CLEAR
1479 v@:dvLastCommand
1480 E*:fResetLog
1481 E*:fLog0
1482 J:lFIN_CMD_PROCESSING
1483 L:lN_CLEAR
1484 F!:^print$:lN_PRINT
1485 V@:dvLastCommand
1486 E*:fGetLatestOutputBuffer
1487 E*:fLog0
1488 J:lFIN_CMD_PROCESSING
1489 L:lN_PRINT
1490 F!:^logout$:lN_LOGOUT
1491 C!@:dvUSERNAME
1492 C*:vUSERNAME
1493 E*:fResetLog
1494 C@:dvLastOutput_backup
1495 E*:fOSHandle
1496 X:{You have SIGNED OUT!
1497 Thanks for Using }
1498 J:lEXIT_Main
1499 L:lN_LOGOUT
1500 F!:^exit$:lN_EXIT
1501 E*:fResetLog
1502 E*:fOSHandle
1503 X:{Thanks for Using }
1504 J:lEXIT_Main
1505 L:lN_EXIT
1506 F!:^export$:lN_EXPORT
1507 Y@:dvLOG
1508 V:vLOG
1509 E*:fResetLog
1510 E*:fOSHandle
1511 X:{Thanks for Using }
1512 V:vMSG
1513 G*!:vDELIM2:vLOG:vMSG
1514 J:lEXIT_Main
1515 L:lN_EXPORT
1516 F!:^{(runnable|register|add-program)\s+([a-z0-9_]+(\.[a-z0-9]+)?)$}:lN_RUNNABLE_FILE
1517 Y:vIN_raw
1518 t!.:
1519 V@:dvLastCommand
1520 d:^\S+\s+
1521 V:vFileName
1522 Y:vFileName
1523 E*:fFileExists
1524 F!:^{^FILE EXISTS$}:lCANNOT_REGISTER_PROG
1525 Y:vFileName
1526 E*:fRegisterProgramFile
```



```

1527 E*:fLog0
1528 J:lFIN_CMD_PROCESSING
1529 L:lCANNOT_REGISTER_PROG
1530 R@:{File #vFileName# does NOT EXIST!
1531 User-Program Registration Operation ABANDONED}
1532 E*:fLog0
1533 J:lFIN_CMD_PROCESSING
1534 L:lN_RUNNABLE_FILE
1535 F!:{~programs$}:lN_LIST_PROGRAMS
1536 V@:dvLastCommand
1537 E*:fShowProgramsList
1538 E*:fLog0
1539 J:lFIN_CMD_PROCESSING
1540 L:lN_LIST_PROGRAMS
1541 F!:{~programs\s+(\S+\s*)+$}:lN_SEARCH_PROGRAMS
1542 Y:vIN_raw
1543 t!.:
1544 V@:dvLastCommand
1545 d:~\S+\s+
1546 V:vFilePattern
1547 E*:fSearchProgramsList
1548 E*:fLog0
1549 J:lFIN_CMD_PROCESSING
1550 L:lN_SEARCH_PROGRAMS
1551 F!:{^(run|exec|execute)\s+.*$}:lN_RUN_FILE
1552 Y:vIN_raw
1553 t!.:
1554 V@:dvLastCommand
1555 d:~\S+\s+
1556 V:vFileName
1557 E*:fRunFile
1558 E*:fLog0
1559 J:lFIN_CMD_PROCESSING
1560 L:lN_RUN_FILE
1561 F!:{^(isrunnable|canrun)\s+.*$}:lN_CANRUN_FILE
1562 Y:vIN_raw
1563 t!.:
1564 V@:dvLastCommand
1565 d:~\S+\s+
1566 V:vFileName
1567 E*:fProgramExists
1568 E*:fLog0
1569 J:lFIN_CMD_PROCESSING
1570 L:lN_CANRUN_FILE
1571 F!:{^(del-prog|delete-program|unregister)\s+([a-z0-9_]+(\.[a-z0-9]+)?)$}:
    lN_UNREGISTER_FILE
1572 Y:vIN_raw
1573 t!.:
1574 V@:dvLastCommand
1575 d:~\S+\s+
1576 V:vFileName
1577 Y:vFileName
1578 E*:fFileExists
1579 F!:{^FILE EXISTS$}:lCANNOT_REGISTER_PROG
1580 Y:vFileName
1581 E*:fUnRegisterProgram
1582 E*:fLog0
1583 J:lFIN_CMD_PROCESSING
1584 L:lCANNOT_REGISTER_PROG
1585 R@:{File #vFileName# does NOT EXIST!
1586 User-Program Unregistration Operation ABANDONED}
1587 E*:fLog0
1588 J:lFIN_CMD_PROCESSING
1589 L:lN_UNREGISTER_FILE
1590 F!:{^(update|update-program)\s+([a-z0-9_]+(\.[a-z0-9]+)?)\s+([a-z0-9_]+(\.[a-z0-9]+)?)$
    }:lN_UPDATE_PROGRAM

```

```

1591 Y:vIN_raw
1592 t!.:
1593 V@:dvLastCommand
1594 d:~\S+\s+
1595 V:vFileNames
1596 Y:vFileNames
1597 D:\s+.*$
1598 V:vFileName
1599 Y:vFileNames
1600 D!\s+.*$
1601 V:vUpdateFileName
1602 Y:vUpdateFileName
1603 E*:fFileExists
1604 F!:{^FILE EXISTS$}:lCANNOT_READ_SRC
1605 Y:vFileName
1606 E*:fFileExists
1607 F!:{^FILE EXISTS$}:lCANNOT_WRITE_TARGET
1608 Y:vFileName
1609 E*:fProgramExists
1610 F:{^PROGRAM EXISTS$}:lPROCESS_UPDATE_PROGRAM
1611 R@:{Target File #vFileName# IS NOT a PROGRAM!
1612 Program UPDATE Operation ABANDONED}
1613 E*:fLog0
1614 J:lFIN_CMD_PROCESSING
1615 L:lPROCESS_UPDATE_PROGRAM
1616 Y:vUpdateFileName
1617 E*:fReadFile
1618 v@:dvDATABUFFER
1619 Y:vFileName
1620 E*:fSaveDataToFile
1621 v:vWRITELOG
1622 R@:{Program File #vFileName# UPDATED USING #vUpdateFileName#
1623 With [WRITE] status: #vWRITELOG#}
1624 E*:fLog0
1625 J:lFIN_CMD_PROCESSING
1626 L:lCANNOT_WRITE_TARGET
1627 R@:{Target File #vFileName# does NOT EXIST!
1628 Program UPDATE Operation ABANDONED}
1629 E*:fLog0
1630 J:lFIN_CMD_PROCESSING
1631 L:lCANNOT_READ_SRC
1632 R@:{Source File #vUpdateFileName# does NOT EXIST!
1633 Program UPDATE Operation ABANDONED}
1634 E*:fLog0
1635 J:lFIN_CMD_PROCESSING
1636 L:lN_UPDATE_PROGRAM
1637 F!:{^([a-z0-9_]+(\.[a-z0-9]+)?)\s*$}:lN_RUN_FILE_PROG
1638 Y:vIN_raw
1639 t!.:
1640 V@:dvLastCommand
1641 V:vFileName
1642 Y:vFileName
1643 E*:fProgramExists
1644 F:{^PROGRAM EXISTS$}:lPROCESS_RUN_PROGRAM:lCMD_CATCHALL
1645 L:lPROCESS_RUN_PROGRAM
1646 Y:vFileName
1647 E*:fRunFile
1648 E*:fLog0
1649 J:lFIN_CMD_PROCESSING
1650 L:lN_RUN_FILE_PROG
1651 L:lCMD_CATCHALL
1652 F:~$:lFIN_CMD_PROCESSING
1653 Y:vIN_raw
1654 V@:dvLastCommand
1655 I!:{--COMMAND NOT FOUND | INVALID COMMAND--}
1656 E*:fLog0

```

```
1657 J:lFIN_CMD_PROCESSING
1658 L:lFIN_CMD_PROCESSING
1659 E*:fLogAI
1660 J:lSTART_Main
1661 L:lEXIT_Main
1662 C!@:dvLOG
1663 Q!:
1664 L:lSYS_SHELL_END
```

Listing 1: TEA Program: TOS v1.1.0 Complete Source-Code (SANITIZED form)